

ECOMMERCE ETL PIPELINE

Technical Documentation

Data Engineering Assignment

Author	Bala
Platform	Databricks (PySpark + Delta Lake)
Stack	Python, SQL, Spark
Date	May 2026

CONTENTS

1 Overview	2
2 Source Files	2
3 Data Model	2
3.1 Dimension Tables	2
3.2 Fact Table	3
4 Data Quality Issues	3
4.1 Unrecoverable Rows (Quarantined)	3
4.2 Fixed Inline	3
5 Analytics Metrics and Dashboard	4
5.1 Daily Active Users by Region	4
5.2 Sweet Category Revenue, Order Count and Quantity	4
5.3 Top 3 Products by Region	4
5.4 Customer Lifetime Value	5
5.5 Duplicate Orders and Faulty Transactions	5
6 Pipeline Notebooks	5
7 Trade-offs and Scale Considerations	6
7.1 What Was Simplified	6
7.2 What Would Change at 100x Scale	6
8 AI Tool Usage	6

OVERVIEW

This pipeline ingests raw ecommerce data from five source files, cleans and validates it, builds a dimensional model, and exposes five core business metrics through a Databricks SQL dashboard.

The pipeline follows a three-layer medallion architecture:

- **Bronze** holds raw data exactly as it appears in the source files.
- **Silver** holds type cast, cleaned, and enriched data.
- **Gold** holds the dimensional model ready for analytics.

SOURCE FILES

File	Description	Notes
customer.csv	4 customers, 3 snapshots each	12 rows total. Conflicting active flags across snapshots.
orders.csv	24 raw order rows	Includes exact duplicates, null customer IDs, and a fractional quantity.
product.csv	14 products	Mixed case currency for Yen products.
prod_cat_tree.csv	5 categories across 2 levels	Used to resolve the top-level category per product.
currency_conversion.json	FX rates for USD, YEN, POUND	Duplicate entries on 2020-01-28. 342-day gap in coverage.

DATA MODEL

The Gold layer uses a star schema with one central fact table and three surrounding dimension tables.

Dimension Tables

Table	Grain	Key Columns
dim_customer	One row per customer	cust_key, region, customer_type, is_active
dim_product	One row per product	prod_key, prod_name, category_name, top_category, price_usd
dim_date	One row per order date	date_key (YYYYMMDD), year, month, quarter, is_weekend

Fact Table

Table	Grain	Key Columns
fact_orders	One row per order line (order + product)	order_id, cust_key, prod_key, date_key, quantity, revenue_usd, status

Revenue is computed as: $price \times FX \text{ rate on the order date} \times quantity$, converted to USD. Multi-line orders are preserved as separate rows. Order A-005 has 2 product lines and A-009 has 3 product lines.

DATA QUALITY ISSUES

Issues are split into two groups: rows that can be fixed with a clear deterministic rule, and rows that cannot be recovered without fabricating data.

Unrecoverable Rows (Quarantined)

Order	Issue	Reason for Quarantine
A-21	No customer ID	Cannot link to any customer, region, or segment.
A-22	No customer ID	Cannot link to any customer, region, or segment.
A-013	Quantity is 0.1	Rounding up or down would fabricate revenue.

Fixed Inline

Issue	Source	Fix Applied
3 snapshots per customer with conflicting active flags	customer.csv	Keep the latest snapshot by created_at date.
Exact duplicate rows for order A-005	orders.csv	Collapse using window deduplication on all columns.
Currency written as <i>Yen</i> instead of YEN	product.csv	Normalize to uppercase.
Duplicate FX rates on 2020-01-28	currency_conversion.js	Keep the last entry per date and currency.
342-day gap in FX rate coverage	currency_conversion.js	Forward fill the last known rate per currency.

ANALYTICS METRICS AND DASHBOARD

All five metrics are computed using SQL against the Gold layer and visualized in the Databricks dashboard.

Daily Active Users by Region

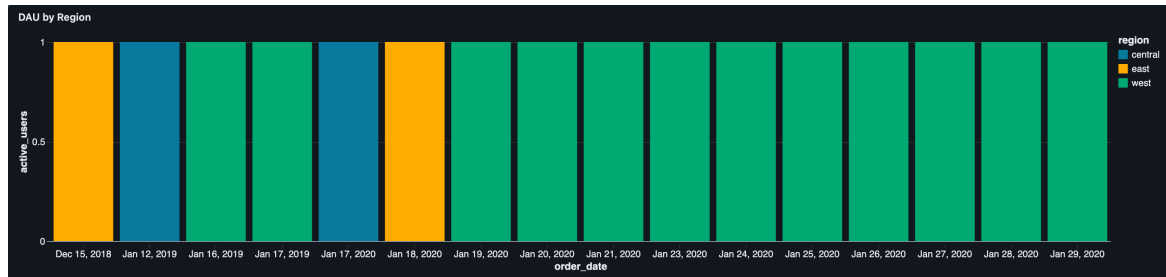


Figure 1: Unique customers who placed an order each day, grouped by region.

Sweet Category Revenue, Order Count and Quantity

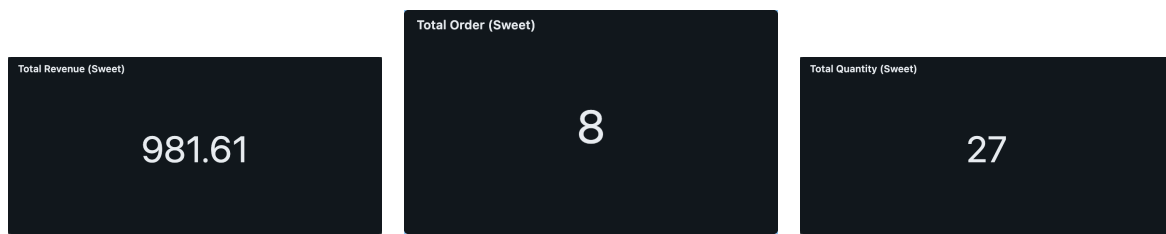


Figure 2: Total revenue, order count, and quantity for the Sweet product category.

Top 3 Products by Region

Top 3 Products by Region				
region	Rank	Product Name	Category	Revenue (USD)
central	1	ffff	Sweet	\$35.47
central	2	cccc	Sweet	\$33.03
central	3	bbbb	Sweet	\$19.82
east	1	dddd	Sweet	\$107.47
east	2	ergertg	Sweet	\$103.44
east	3	asdfdf	Sweet	\$78.16
west	1	kjhijk	crisp	\$2.24342K
west	2	ruy5u	chip	\$951.39
west	3	dafsfdf	Sweet	\$216.31

Figure 3: Top 3 products ranked by total revenue within each region.

Customer Lifetime Value

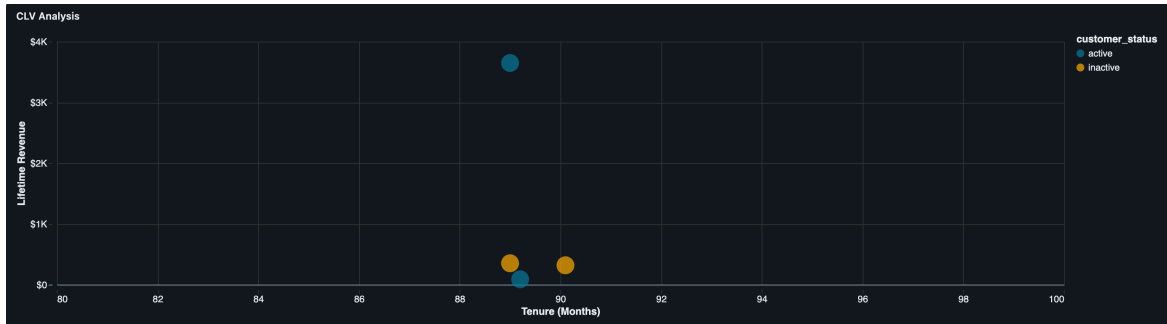


Figure 4: Lifetime revenue plotted against customer tenure in months, segmented by active and inactive status.

Duplicate Orders and Faulty Transactions

Flag Type	Order ID	Line Count	Product Keys	Total Revenue (USD)	Error Reason
INVALID_QTY	A-013	1	null	null	Quantity is fractional (0.1). Rounding up or down would fabricate revenue. Excluded from all revenue metrics.
NULL_CUST_ID	A-21	1	null	null	No customer ID. Order cannot be linked to any customer, region, or segment. Excluded from all business metrics.
NULL_CUST_ID	A-22	1	null	null	No customer ID. Order cannot be linked to any customer, region, or segment. Excluded from all business metrics.
multi_line_order	A-005	2	[3,2]	\$52.85	null
multi_line_order	A-009	3	[4,11,7]	\$289.07	null

Figure 5: Quarantined error rows and multi-line orders shown in one audit view.

PIPELINE NOTEBOOKS

Notebook	Purpose
main_pipeline	Entry point. Loads all notebooks and runs the pipeline end to end.
00_config	All constants in one place including paths, table names, and schema names.
etl_utils	Shared helper functions used across every notebook.
01_schema	Creates all Bronze, Silver, Gold, Error, and Audit table schemas.
02_read_data	Reads all source files into DataFrames using explicit schemas.
03_data_cleaning	Deduplicates, validates, and splits rows into clean vs quarantined.
04_data_transformations	Casts types, applies business rules, and enriches with FX and category data.
05_dim_transforms	Builds dim_customer, dim_product, and dim_date.
06_fact_transforms	Builds fact_orders by joining order items to the date dimension.

TRADE-OFFS AND SCALE CONSIDERATIONS

What Was Simplified

- The pipeline uses overwrite mode on every table. This is simple and repeatable but does not support incremental updates.
- The customer dimension holds one snapshot per customer with no SCD Type 2 (Slowly Changing Dimension) basically no need to keep a history tracking.
- Foreign exchange rate gaps are forward filled, assuming the rate held steady across missing dates.
- No orchestration tool is used. The pipeline runs as a single notebook with no scheduling or retry logic.

What Would Change at 100x Scale

- Switch from overwrite to MERGE using Delta Lake for incremental processing.
- Add SCD Type 2 to the customer dimension to track changes over time.
- Introduce Databricks Workflows or Apache Airflow for scheduling or even ADF, retries, and dependency management.
- Partition fact_orders by order date to improve query performance on large datasets.
- Add row count trend monitoring to catch unexpected drops between pipeline runs.

AI TOOL USAGE

Claude was used to assist with code review, preparing this documentation through overleaf and certain contents and coding parts. All pipeline logic, SQL queries, and design decisions or system design were done and validated by the me.